

Plan Switch Detection

Part 1: How and Why Plan Switches Occur

In a relational database management system (RDBMS) like PostgreSQL, the Query Optimizer evaluates various possible execution strategies (query plans) and selects the one with the lowest estimated execution cost.

A **Plan Switch** occurs when a change in the query's parameter (here, the `[DATE]`) alters the statistical distribution of the data being requested, causing the optimizer to abandon one strategy in favor of another.

The Mechanics: Selectivity and Cardinality

The driving force behind these switches is **Selectivity**. Selectivity represents the estimated fraction of table rows that will satisfy a query's predicate.

1. **High Selectivity** (e.g., `l_shipdate <= '1992-01-02'`): The predicate matches very few rows. The optimizer expects a low output cardinality.
2. **Low Selectivity** (e.g., `l_shipdate <= '1998-01-01'`): The predicate matches millions of rows, returning a massive portion of the table.

The Cost Model: Why Plans Change

PostgreSQL relies on mathematical cost formulas for different physical operators. As selectivity changes, the optimal operator changes:

- **Index Scans / Bitmap Scans:** These are incredibly fast for retrieving a small number of rows. However, as the number of requested rows grows, the cost skyrockets due to "random I/O"—the database has to constantly jump between index pages and table data pages.
- **Sequential (Seq) Scans:** When a query requests a large portion of the table, an index becomes a liability. The optimizer calculates that it is cheaper to simply read the entire table from disk sequentially and filter out the non-matching rows in memory.
- **Join Strategies (Nested Loop vs. Hash Join):** For small datasets, looping through rows one by one (Nested Loop) is efficient. As the outer table grows, repeated lookups become too expensive, and the optimizer switches to building a Hash table in memory (Hash Join).
- **Parallel Processing (Gather nodes):** As the estimated data volume increases, the optimizer may decide that the computational cost of aggregations or scans justifies the overhead of spinning up background worker processes.

Part 2: Detected Plan Switches by Query

Query 1 Analysis:

Context: Query 1 performs heavy aggregations on the `lineitem` table based on the predicate `l_shipdate <= [DATE]`. As the date advances, the volume of data being scanned and aggregated grows massively.

Context: This query performs aggregation on the `lineitem` table. As the date advances, the volume of data scanned increases.

Plan Switch 1: 1992-01-07 and 1992-01-08

- **Plan Before:**

```
Aggregate (Plain)
  Bitmap Heap Scan on lineitem
    Bitmap Index Scan using idx_lineitem_shipdate
```

- **Plan After:**

```
Aggregate (Plain)
  Index Scan on lineitem using idx_lineitem_shipdate
```

- **Why it happens:** In the very early dates, the dataset is so small that the optimizer calculates a direct Index Scan is slightly cheaper than the overhead of building a memory bitmap first.

Plan Switch 2: 1992-01-27 and 1992-01-28

- **Plan Before:**

```
Aggregate (Plain)
  Index Scan on lineitem using idx_lineitem_shipdate
```

- **Plan After:**

```
Aggregate (Plain)
  Gather
    Aggregate (Plain)
      Bitmap Heap Scan on lineitem
        Bitmap Index Scan using idx_lineitem_shipdate
```

- **Why it happens:** The data volume hits the threshold where parallel execution becomes viable. The optimizer spins up worker processes (**Gather**) and reverts to a **Bitmap Heap Scan**, which partitions well among parallel workers.

Plan Switch 3: 1992-03-29 and 1992-03-30

- **Plan Before:**
 - Aggregate (Plain)
 - Gather
 - Aggregate (Plain)
 - Bitmap Heap Scan on lineitem
 - Bitmap Index Scan using idx_lineitem_shipdate
- **Plan After:**
 - Aggregate (Plain)
 - Gather
 - Aggregate (Plain)
 - Seq Scan on lineitem
- **Why it happens:** The classic index tipping point. By late March, the query requests too large a fraction of the table. Random I/O is abandoned in favor of reading the whole table sequentially from disk.

Query 2 Analysis

Context: Query 2 utilizes an **EXISTS** subquery. The parameter (**o_orderdate < [DATE]**) dictates the number of **orders** that need to be evaluated against the **lineitem** table.

Plan Switch 1: 1992-01-01 and 1992-01-02

- **Plan Before:**
 - Aggregate (Sorted)
 - Sort
 - Nested Loop [Semi]
 - Index Scan on orders using idx_orders_orderdate
 - Index Scan on lineitem using lineitem_pkey
- **Plan After:**
 - Aggregate (Sorted)
 - Gather Merge

```

Aggregate (Sorted)
  Sort
    Nested Loop [Semi]
      Bitmap Heap Scan on orders
        Bitmap Index Scan using idx_orders_orderdate
          Index Scan on lineitem using lineitem_pkey

```

- **Why it happens:** A tiny increase in matched rows triggers the optimizer to deploy parallel workers (**Gather Merge**). To feed these workers efficiently, the access method on **orders** upgrades from a standard Index Scan to a Bitmap Heap Scan.

Plan Switch 2: 1992-05-13 and 1992-05-14

- **Plan Before:**

```

Aggregate (Sorted)
  Gather Merge
    Aggregate (Sorted)
      Sort
        Nested Loop [Semi]
          Bitmap Heap Scan on orders
            Bitmap Index Scan using idx_orders_orderdate
              Index Scan on lineitem using lineitem_pkey

```

- **Plan After:**

```

Aggregate (Sorted)
  Gather Merge
    Aggregate (Sorted)
      Sort
        Hash Join [Semi]
          Seq Scan on orders
            Hash
              Seq Scan on lineitem

```

- **Why it happens:** A major structural change. The outer relation (**orders**) is now yielding so many rows that a Nested Loop lookup is too slow. The optimizer switches to sequentially scanning both tables and joining them via an in-memory Hash Join.

Plan Switch 3: 1997-11-25 and 1997-11-26

- **Plan Before:**

```
Aggregate (Sorted)
  Gather Merge
    Aggregate (Sorted)
      Sort
        Hash Join [Semi]
          Seq Scan on orders
            Hash
              Seq Scan on lineitem
```

- **Plan After:**

```
Aggregate (Sorted)
  Gather Merge
    Sort
      Aggregate (Hashed)
        Hash Join [Semi]
          Seq Scan on orders
            Hash
              Seq Scan on lineitem
```

- **Why it happens:** Instead of sorting a massive joined dataset to aggregate it, the optimizer decides it is cheaper to use Hash Aggregation on the raw output first, and then sort the much smaller aggregated result set.

Query 3 Analysis

Context: Query 3 is a three-way join between `customer`, `orders`, and `lineitem`. It has diverging predicates: `o_orderdate < [DATE]` (selectivity decreases over time) and `l_shipdate > [DATE]` (selectivity *increases* over time).

As the date parameter advances, the number of qualifying rows in the `orders` and `lineitem` tables grows, causing severe volatility in the execution plan during the early months of 1992.

Plan Switch 1: 1992-01-01 and 1992-01-02

- **Plan Before:**

```
Nested Loop [Inner]
```

```
Nested Loop [Inner]
  Index Scan on orders using idx_orders_orderdate
  Index Scan on customer using customer_pkey
  Index Scan on lineitem using lineitem_pkey
```

- **Plan After:**

```
Gather
  Nested Loop [Inner]
    Nested Loop [Inner]
      Bitmap Heap Scan on orders
        Bitmap Index Scan using idx_orders_orderdate
      Index Scan on customer using customer_pkey
    Index Scan on lineitem using lineitem_pkey
```

- **How and Why it Happens:** For the very first day of the year, the dataset is tiny, and a purely sequential, nested loop approach using standard index scans is cheapest. On January 2nd, the matched row count ticks over a threshold. The optimizer decides the data volume now justifies parallel processing (**Gather**). Simultaneously, the access method on **orders** upgrades from a standard **Index Scan** to a **Bitmap Heap Scan** to handle the incoming rows more efficiently across parallel workers.

Plan Switch 2: 1992-01-03 and 1992-01-04

- **Plan Before:**

```
Gather
  Nested Loop [Inner]
    Nested Loop [Inner]
      Bitmap Heap Scan on orders
        Bitmap Index Scan using idx_orders_orderdate
      Index Scan on customer using customer_pkey
    Index Scan on lineitem using lineitem_pkey
```

- **Plan After:**

```
Gather
  Nested Loop [Inner]
    Hash Join [Inner]
      Seq Scan on customer
      Hash
        Bitmap Heap Scan on orders
```

```
Bitmap Index Scan using idx_orders_orderdate
Index Scan on lineitem using lineitem_pkey
```

- **How and Why it Happens:** The join strategy between `customer` and `orders` changes. As the `orders` dataset continues to grow, fetching rows one-by-one from `customer` via a `Nested Loop` becomes too expensive. The optimizer switches to a `Hash Join`, sequentially scanning `customer` and hashing the `orders` dataset in memory for much faster batch lookups.

Plan Switch 3: 1992-01-06 and 1992-01-07

- **Plan Before:**

```
Gather
  Nested Loop [Inner]
    Hash Join [Inner]
      Seq Scan on customer
      Hash
        Bitmap Heap Scan on orders
          Bitmap Index Scan using idx_orders_orderdate
          Index Scan on lineitem using lineitem_pkey
```

- **Plan After:**

```
Gather
  Nested Loop [Inner]
    Hash Join [Inner]
      Bitmap Heap Scan on orders
        Bitmap Index Scan using idx_orders_orderdate
      Hash
        Seq Scan on customer
      Index Scan on lineitem using lineitem_pkey
```

- **How and Why it Happens:** The optimizer swaps the roles of the tables inside the `Hash Join`. Instead of hashing `orders`, it now hashes `customer` and scans `orders` to probe the hash table. This happens because the optimizer re-estimates the sizes of the filtered datasets and decides that the `customer` table is now the better candidate to fit entirely into the in-memory hash table.

Plan Switch 4: 1992-01-18 and 1992-01-19

- **Plan Before:**

```
Gather
  Nested Loop [Inner]
    Hash Join [Inner]
      Bitmap Heap Scan on orders
        Bitmap Index Scan using idx_orders_orderdate
      Hash
        Seq Scan on customer
    Index Scan on lineitem using lineitem_pkey
```

- **Plan After:**

```
Gather
  Hash Join [Inner]
    Hash Join [Inner]
      Seq Scan on lineitem
    Hash
      Bitmap Heap Scan on orders
        Bitmap Index Scan using idx_orders_orderdate
  Hash
    Seq Scan on customer
```

- **How and Why it Happens:** The outer join to `lineitem` changes from a `Nested Loop` to a `Hash Join`. By mid-January, the combined result set of `customer` + `orders` is so large that doing individual index lookups into `lineitem` for every single matched combination is slower than simply reading the entire `lineitem` table. `lineitem` is now sequentially scanned.

Plan Switch 5: 1992-02-14 and 1992-02-15

- **Plan Before:**

```
Gather
  Hash Join [Inner]
    Hash Join [Inner]
      Seq Scan on lineitem
    Hash
      Bitmap Heap Scan on orders
```

```
Bitmap Index Scan using idx_orders_orderdate
Hash
Seq Scan on customer
```

- **Plan After:**

```
Gather
Hash Join [Inner]
Hash Join [Inner]
Seq Scan on lineitem
Hash
Seq Scan on orders
Hash
Seq Scan on customer
```

- **How and Why it Happens:** The access method for the `orders` table crosses its tipping point. The date filter now captures enough of the table that reading `orders` sequentially from disk (`Seq Scan`) is cheaper than suffering the random I/O overhead of a `Bitmap Heap Scan`.

Plan Switch 6: 1992-03-08 and 1992-03-09

- **Plan Before:**

```
Gather
Hash Join [Inner]
Hash Join [Inner]
Seq Scan on lineitem
Hash
Seq Scan on orders
Hash
Seq Scan on customer
```

- **Plan After:**

```
Gather
Hash Join [Inner]
Seq Scan on lineitem
Hash
Hash Join [Inner]
Seq Scan on orders
```

```
Hash
  Seq Scan on customer
```

- **How and Why it Happens:** The execution tree shape reorganizes. Instead of joining `lineitem` to `orders` first, the optimizer determines it is now more efficient to join `orders` and `customer` first, hash that intermediate result set, and then probe it using the `lineitem` table. This indicates shifting cardinality estimates making the `orders+customer` intermediate relation smaller.

Plan Switch 7: 1992-03-13 and 1992-03-14

- **Plan Before:**

```
Gather
  Hash Join [Inner]
    Seq Scan on lineitem
    Hash
      Hash Join [Inner]
        Seq Scan on orders
        Hash
          Seq Scan on customer
```

- **Plan After:**

```
Gather
  Hash Join [Inner]
    Seq Scan on lineitem
    Hash
      Hash Join [Inner]
        Bitmap Heap Scan on orders
          Bitmap Index Scan using idx_orders_orderdate
        Hash
          Seq Scan on customer
```

- **How and Why it Happens:** A regression! The optimizer switches the access method on `orders` *back* to a `Bitmap Heap Scan` from a `Seq Scan`. This volatility occurs when the estimated costs of a Seq Scan and Bitmap Scan are nearly identical. Tiny daily changes in data distribution (or statistics thresholds) cause the optimizer to momentarily flip back to its previous logic.

Plan Switch 8: 1992-05-16 and 1992-05-17

- **Plan Before:**

```
Gather
  Hash Join [Inner]
    Seq Scan on lineitem
      Hash
        Hash Join [Inner]
          Bitmap Heap Scan on orders
            Bitmap Index Scan using idx_orders_orderdate
          Hash
            Seq Scan on customer
```

- **Plan After:**

```
Gather
  Hash Join [Inner]
    Seq Scan on lineitem
      Hash
        Hash Join [Inner]
          Seq Scan on orders
          Hash
            Seq Scan on customer
```

- **How and Why it Happens:** The `orders` access method flips *back* to a `Seq Scan` permanently. By mid-May, the date parameter encompasses a massive block of data. The `Seq Scan` firmly and definitively beats the `Bitmap Heap Scan` in the optimizer's cost estimates, and the plan finally settles into this stable configuration.

Summary:

Query 1:

This query's journey is a straightforward battle of table access methods as the date range widens.

[Initial Plan] Before Jan 8, 1992

- Plain Aggregate + Bitmap Heap Scan



[Switch 1] 1992-01-08

- Trigger: Dataset is small enough that a direct index is cheaper.
- New Plan: Index Scan



[Switch 2] 1992-01-28

- Trigger: Data volume justifies worker processes.
- New Plan: Add Gather (Parallelism) + revert to Bitmap Heap Scan



[Switch 3] 1992-03-30

- Trigger: The tipping point. Too much data for random index I/O.
- New Plan: Seq Scan



[Final Plan]

- Gather + Aggregate + Seq Scan (lineitem)

Query 2:

This query evolves from row-by-row nested loops to heavy-duty batch processing.

[Initial Plan] Before Jan 2, 1992

- Sorted Aggregate + Nested Loop + Index Scans



[Switch 1] 1992-01-02

- Trigger: Rows increase; time to parallelize the index lookups.
- New Plan: Add Gather Merge + Bitmap Heap Scan (orders)



[Switch 2] 1992-05-14

- Trigger: Nested Loops become too slow for the volume.
- New Plan: Hash Join + Seq Scans (orders & lineitem)



[Switch 3] 1997-11-26

- Trigger: Massive data volume; sorting before aggregating is too expensive.
- New Plan: Hashed Aggregate (Hash first, then Sort)



[Final Plan]

- Gather Merge + Hashed Aggregate + Hash Join + Seq Scans

Query 3:

This is the most volatile query. Notice how it slowly replaces Nested Loops with Hash Joins, and then spends months flip-flopping between scan types as the data slowly grows.

[Initial Plan] Before Jan 2, 1992

- Pure Nested Loops + Index Scans across all 3 tables



[Switch 1] 1992-01-02

- Trigger: Add Parallelism.
- New Plan: Add Gather + Bitmap Heap Scan (orders)



[Switch 2] 1992-01-04

- Trigger: Nested loop to 'customer' is too slow.
- New Plan: Hash Join (orders & customer)



[Switch 3] 1992-01-07

- Trigger: Optimizer decides 'customer' is smaller to keep in memory.
- New Plan: Swap Hash sides (Hash customer, probe with orders)



[Switch 4] 1992-01-19

- Trigger: Outer join to 'lineitem' gets too big for nested loops.

- New Plan: Massive Hash Join + Seq Scan (lineitem)



[Switch 5] 1992-02-15

- Trigger: 'orders' table crosses the Seq Scan tipping point.
- New Plan: Seq Scan (orders)



[Switch 6] 1992-03-09

- Trigger: Better join ordering discovered.
- New Plan: Reorganize tree (Join orders+customer first, then lineitem)



[Switch 7] 1992-03-14

- Trigger: Regression! Data stats shift slightly.
- New Plan: Revert 'orders' back to Bitmap Heap Scan



[Switch 8] 1992-05-17

- Trigger: 'orders' table is permanently too large for indexes.
- New Plan: Finalize Seq Scan (orders)



[Final Plan]

- Gather + Nested Hash Joins + pure Seq Scans on all tables